

Programming Project / C++

Auto Rental Fleet Management System

Student 1: Simion Sirb

Student 2: Csiszer Mihai

I. Task Description

A **car rental fleet management system** implemented in C++17 with a full-screen interactive terminal interface. Two separate executables communicate through shared text files. All user interaction is done through an arrow-key navigable menu — no `std::cin` or command-line arguments are used.

Student 1 – Fleet Manager (app_1)

- View all vehicles in the fleet
- Add, delete, and modify vehicles
- Search vehicles by brand
- View all existing reservations

Student 2 – Customer Portal (app_2)

- View currently available vehicles
 - Make a reservation for a date range
 - Cancel an existing reservation
 - View own reservations by customer ID
-

II. Data Structures & OOP Design

Car inherits **Vehicle** (inheritance). **Reservation** contains a **Car** object by value (composition). Both classes override **operator<<** for stream output.

Class	Fields & Notes
Vehicle (abstract base)	string id, brand, vehicleType bool available int seats Pure virtual: toFileLine(), display() operator<< dispatches to display()
Car (extends Vehicle)	string fuelType double pricePerDay toDisplay() – one-line UI summary operator== (by ID) operator< (by price) static fromFileLine() – deserialisation

Class	Fields & Notes
Date	int day, month, year operator<< / >> operator< / <= / == isValid(), toString()
Reservation (composition)	string reservationId, customerId Car car (owned by value) Date startDate, endDate toDisplay(), toFileLine()

III. File Structure

Both applications share two plain-text files in the working directory and reload them before every operation to stay in sync.

flota.txt

Stores all vehicles:

```
<numar de masini>
<id> <marca> <tip> <disponibil> <locuri> <combustibil> <pret/zi>
...
```

rezervari.txt

Stores all reservations, one per line:

```
<idRezervare> <idClient> <idMasina> <ziStart> <lunaStart> <anStart> <ziEnd>
<lunaEnd> <anEnd>
...
```

IV. Menu Screens

Application 1 – Fleet Manager (app_1)

Option	Description
View all vehicles	Displays all vehicles with ID, brand, type, seats, fuel, price and availability
Add vehicle	Form: Brand, Type, Seats, Fuel, Price/day. ID is generated automatically.
Delete vehicle	Prompts for vehicle ID and removes it from the fleet
Modify vehicle	Prompts for ID, then sub-menu: Price / Availability / Seats / Fuel / Brand
Search vehicle by brand	Prompts for a keyword and filters vehicles (case-insensitive)
View all reservations	Reads rezervari.txt and displays all bookings
Quit	Exits the application

Application 2 – Customer Portal (app_2)

Option	Description
View available vehicles	Lists only vehicles where available = true
Make a reservation	Form: Customer ID, Car ID, Start date, End date. Validates dates and availability. Marks car as rented.
Cancel a reservation	Prompts for Reservation ID. Removes the reservation and marks the car as available again.
View my reservations	Prompts for Customer ID and filters all matching reservations
Quit	Exits the application

V. Graphical Interface – FTXUI

The interactive terminal UI is built with **FTXUI** (Functional Terminal UI), a pure C++17 library that renders full-screen interfaces using standard terminal escape codes. It requires no external dependencies beyond a C++17 compiler and is fetched automatically at build time via CMake FetchContent.

How it works

FTXUI uses an **event loop** model. Each screen is created with `ScreenInteractive::Fullscreen()`, which takes full control of the terminal. The app calls `screen.Loop(component)`, which blocks and redraws on every keypress. Exiting is done via `screen.ExitLoopClosure()()`.

UI Components used

Component	Purpose
<code>Menu(&items;, &selected;)</code>	Arrow-key navigable list. Used for the main menu and all list views.
<code>Input(&value;, placeholder, opt)</code>	Single-line text field. Used in forms to collect IDs, dates, and prices.
<code>Button(label, callback)</code>	Executes a lambda on Enter. Used for OK / Cancel in every form and message box.
<code>Container::Vertical({...})</code>	Stacks components vertically and manages Tab-key focus inside a form.
<code>Container::Horizontal({...})</code>	Places OK and Cancel buttons side by side.
<code>Renderer(component, fn)</code>	Wraps a component and defines its visual layout via an element tree lambda.
<code>CatchEvent(renderer, fn)</code>	Intercepts raw key events. Used to handle Enter, Escape, Q, and arrow keys.

Three reusable UI helpers

Function	Description
showList(title, items)	Full-screen scrollable read-only list. Press Enter or Esc to return. Used by all view commands.
showForm(title, fields)	Full-screen form with one Input per field and OK / Cancel buttons. Returns true if confirmed.
msgBox(message, isError)	Small centered dialog with green INFO or red ERROR header and an OK button.

Keyboard navigation

Key	Action
↑ / ↓	Move the highlighted option up or down in a menu
Enter	Confirm the selected option or submit a form
Tab	Move focus to the next input field inside a form
Esc / Q	Close the current screen and return to the previous menu